Project Atlas — Project Evaluation Report

Evaluation type: Evidence-led engineering review
Project: Project Atlas
Report status: Preliminary — local execution required to complete runtime validation
Prepared: 19 October 2026

Evidence note: Prior project history records a VS Code Agent review in which 28 backend tests passed and `git diff --check` passed. Those outputs were not available for independent rerun in this chat, and the local repository, database, running services, and synthetic fixtures are not mounted here. That historical result is therefore not counted as a fresh execution. Checks requiring local access are marked BLOCKED or NOT RUN rather than presented as successful.

1. Project Overview

Project Atlas is a web application intended to support report-related questions and calculations through a structured backend workflow. Based on available project notes, its components include:

- A web frontend served locally under `/app/`
- A Python FastAPI backend
- MySQL for structured client/report-related data
- Chroma for retrieval-related functionality
- A modular workflow with Router, Retriever, Calculator, Critic, Writer, and Escalation Guard components
- Workspace login/session handling and client-scoped request handling
- Synthetic data and test fixtures intended for safe evaluation

Technologies identified in project notes: Python, FastAPI, Uvicorn, MySQL, Chroma, pytest, and a browser-based frontend. Confirm the exact frontend framework and dependency versions against the current repository before final submission.

2. Evaluation Objectives

This evaluation aims to verify that the application:

1. Starts and passes its existing automated test suite.
2. Handles workspace authentication and session state safely.

3. Validates client identifiers and enforces client/report access boundaries.

4. Retrieves relevant information and grounds answers in available sources.

5. Performs supported calculations, including Europe market revenue CAGR from 2020 to 2025 using the synthetic dataset.

6. Handles missing evidence, malformed requests, unsupported uploads, and size limits safely.

7. Returns controlled API errors and communicates failures clearly in the frontend.

8. Loads synthetic data repeatably and protects existing data during cleanup.

9. Has a reproducible test process suitable for reviewer inspection.

3. Testing Methodology

Environment and evidence available

Item	Available evidence
Operating environment	Windows, local VS Code workspace (from project history)
Python environment	`.venv` in project root
Backend	FastAPI application; previously run with Uvicorn
Database	MySQL database named `atlas\_copilot` in prior project notes
Retrieval	Chroma, according to project notes
Test framework	pytest, according to prior test review
Synthetic data	Synthetic dataset implementation/fixtures are reported in project notes
Direct access in this evaluation	Unavailable; local tests could not be executed here

Commands for local verification

The following are recommended commands, **not represented as executed in this report**:

```powershell
.\\.venv\\Scripts\\python.exe -m pytest backend/tests -q
```

git diff --check
git status --short
```

Before running any dataset loader or cleanup script, inspect its implementation and options. Use only the synthetic/test environment, avoid destructive cleanup, and do not print environment-file contents or secrets.

Status definitions

- **PASS:** Executed and observed behavior met the expectation.
- **FAIL:** Executed and observed behavior did not meet the expectation.
- **BLOCKED:** Could not safely or reliably execute because a required local service, fixture, credential, or repository access was unavailable.
- **NOT RUN:** No execution evidence is available; the test has not been performed for this report.

4. Evaluation Results

This table distinguishes a previously reported test-suite result from tests not independently executed for this report.

| Test ID | Test scenario | Expected result | Actual result | Status | Short remarks |
|---|---|---|---|---|---|
| AT-001 | Existing backend test suite | Existing tests complete without failures | Prior VS Code Agent review reported 28 backend tests passed; not rerun here | BLOCKED | Historical result only; rerun locally |
| AT-002 | Whitespace/conflict-marker validation | `git diff --check` exits successfully | Prior VS Code Agent review reported it passed; not rerun here | BLOCKED | Historical result only |
```

AT-003	Workspace login with valid credentials	Authorized workspace user can log in	No fresh request/response evidence available	BLOCKED	Requires local app and test credentials
AT-004	Workspace login with invalid credentials	Login is rejected without exposing secrets	No fresh request/response evidence available	NOT RUN	Negative case unverified
AT-005	Session persistence and sign-out	Session behavior is consistent; sign-out invalidates session	Prior code-review notes indicate in-memory sessions; no runtime retest	BLOCKED	Restart/multi-worker behavior unverified
AT-006	Valid client ID	Existing authorized client can be accessed	No fresh API response available	BLOCKED	Requires local database and synthetic fixtures
AT-007	Invalid/nonexistent client ID	Request is rejected with a controlled client error	Prior code-review notes say unknown ID returns HTTP 404; not rerun here	BLOCKED	Historical code-review finding only
AT-008	Cross-client data isolation	Client A cannot access Client B's data without authorization	No adversarial test executed	NOT RUN	High-priority authorization test
AT-009	Report retrieval/source grounding	Retrieved answer reflects available	No fixture/output pair available	BLOCKED	Retrieval relevance and grounding unverified

		source material			
AT-010	Europe market revenue CAGR, 2020–2025	Result matches fixture values and CAGR formula	Fixture values and actual application output unavailable here	BLOCKED	Do not state a numeric result until verified
AT-011	Missing/insufficient source data	Application indicates insufficient evidence or safely declines	No fresh request/response evidence available	NOT RUN	Distinguish expected rejection from defect
AT-012	Malformed request/missing fields	Request is rejected with a controlled 4xx response	No fresh request/response evidence available	NOT RUN	Validate schema and error body locally
AT-013	Unsupported upload type	Upload is rejected safely	No upload request executed	BLOCKED	Requires local endpoint and safe test file
AT-014	Upload size limit, including absent Content-Length	Oversized body is rejected before excessive processing	Prior code-review notes identify a possible gap when Content-Length is absent; no test run	BLOCKED	Potential risk; suspected until reproduced
AT-015	API exception/frontend error display	Failure is visible and does not leave	Prior code-review notes indicate sign-out UI may hide itself if	BLOCKED	Potential frontend/session

		misleading UI state	the server request fails; no runtime retest		consistency issue
AT-016	Synthetic dataset loading/repeated execution	Loading is repeatable and does not corrupt or duplicate data	No loader executed in this chat	BLOCKED	Inspect loader and use isolated environment
AT-017	Synthetic cleanup safeguards	Cleanup only removes intended synthetic artifacts	No cleanup command executed	NOT RUN	Do not run destructive cleanup against working DB
AT-018	Session storage across process restart/multiple workers	Session validity is predictable across deployment topology	Prior code-review notes indicate process-local in-memory session storage	BLOCKED	Deployment behavior not runtime tested

Quantitative summary

For the **18 evaluation rows in this report**:

Status	Count
PASS	0
FAIL	0
BLOCKED	12
NOT RUN	6

Total	**18**

Pass percentage for this report: N/A (0 executed evaluation cases). The earlier report of 28 passing backend tests is historical context and is not added to these 18 rows because the output could not be independently verified here. Blocked and unrun cases are not counted as passes. This is a reporting limitation, not a claim that the application has zero passing tests.

5 Failure Analysis and Negative Test Scenarios

No local tests could be executed in this chat. The items below are **risk-based negative scenarios and code-review leads**, not five confirmed runtime failures. Correctly rejecting invalid input is expected behavior; it is a defect only if observed behavior violates the expectation.

FA-01 — Cross-client authorization boundary

- **Scenario:** An authenticated workspace session requests a different numeric `client\_id`.
- **Expected:** The server verifies the caller's authorization for that client and denies access when unauthorized.
- **Actual/evidence:** No adversarial runtime test was performed. Prior code-review notes indicate workspace login is shared and does not bind a user to a particular client; the chat route accepts a numeric client ID and checks client/report records.
- **Root cause: SUSPECTED** authorization gap if deployment requires client-level access separation. Shared workspace authentication alone does not establish per-client authorization.
- **Impact/severity:** High if exposed to multiple untrusted tenants; lower for a strictly local single-user demo.
- **Corrective action:** Enforce a server-side user-to-client authorization check on every client-scoped request.
- **Retest criteria:** A user authorized for Client A can access A but is denied Client B, including via direct API calls and guessed IDs.

FA-02 — Upload limit when Content-Length is absent

- **Scenario:** Send an oversized upload without a trustworthy `Content-Length` header.
- **Expected:** Request size is bounded before the server buffers or parses an untrusted body.
- **Actual/evidence:** No request was sent. Prior code-review notes identify a possible bypass when `Content-Length` is absent and multipart parsing may occur before a bounded read.
- **Root cause: SUSPECTED** reliance on the header or a size check after body parsing.
- **Impact/severity:** High for a publicly reachable service if confirmed; resource exhaustion may be possible.
- **Corrective action:** Enforce request-body limits at the application/server or reverse-proxy layer and count bytes before accepting the full body.
- **Retest criteria:** Oversized bodies with correct, missing, and misleading length headers are all rejected without unbounded buffering.

FA-03 — Process-local session storage

- **Scenario:** Log in, then restart the process or route a request to a different worker.
- **Expected:** Session behavior matches the documented deployment design; sessions are shared across workers or explicitly invalidated and handled gracefully.
- **Actual/evidence:** No restart/multi-worker test was run. Prior code-review notes indicate sessions are stored in an in-memory dictionary.
- **Root cause: CONFIRMED BY PRIOR CODE REVIEW** as an architectural observation; runtime impact is not confirmed in this report.
- **Impact/severity:** Medium for a single-process demo; potentially high for multi-worker or horizontally scaled deployment.
- **Corrective action:** Use a shared session store or a securely designed signed-session mechanism with appropriate expiry and revocation.
- **Retest criteria:** Verify behavior across restart and multiple workers, including expiry and logout invalidation.

FA-04 — Sign-out UI/server state mismatch

- **Scenario:** The sign-out request fails because the server is unavailable or returns an error.
- **Expected:** The UI communicates the failure and accurately reflects whether the server session remains active.
- **Actual/evidence:** No frontend interaction was run. Prior code-review notes indicate the frontend may hide the signed-in UI even if the server sign-out request fails.
- **Root cause: SUSPECTED** optimistic client-side state change without confirming server response.
- **Impact/severity:** Medium; user may believe the session has ended when it has not.
- **Corrective action:** Handle failed sign-out requests explicitly; do not claim server-side logout succeeded until confirmed.
- **Retest criteria:** Simulate a failed sign-out response and verify the UI shows an accurate state; confirm successful sign-out invalidates the server session.

FA-05 — Invalid client ID handling

- **Scenario:** Submit a malformed, negative, or nonexistent client ID.
- **Expected:** Input is validated and rejected with a controlled 4xx response without leaking internal details.
- **Actual/evidence:** Prior code-review notes report HTTP 404 for an unknown ID. No fresh test was run. A 404 for a nonexistent client may be correct behavior, not a defect.
- **Root cause: NOT ESTABLISHED**; the API contract and malformed-ID behavior require runtime verification.
- **Impact/severity:** Low to medium unless malformed values cause exceptions or information leakage.
- **Corrective action:** Document the contract and validate type/range before database access.
- **Retest criteria:** Test valid, nonexistent, negative, string, null, and omitted IDs; confirm predictable status codes and non-sensitive error bodies.

FA-06 — CAGR result and source grounding

- **Scenario:** Ask for Europe market revenue CAGR from 2020 through 2025.
- **Expected:** The system uses the correct beginning and ending values from the synthetic dataset and calculates `((ending\_value / beginning\_value) \*\* (1 / number\_of\_years) - 1) \* 100`, where the period is five years.
- **Actual/evidence:** No fixture values or live response were available to verify a numeric result.
- **Root cause: NOT ESTABLISHED**; no observed calculation failure is claimed.
- **Impact/severity:** High for report correctness if a wrong value is presented as sourced.
- **Corrective action:** Add a deterministic unit test using known synthetic values and assert the numeric result and source/value provenance.
- **Retest criteria:** Independently calculate the expected value from the fixture and compare it with the API response within a documented tolerance; verify missing endpoints produce an insufficiency response.

FA-07 — Missing or insufficient source data

- **Scenario:** Request a metric or period absent from available source data.
- **Expected:** The system states evidence is insufficient and does not invent a result or unsupported citation.
- **Actual/evidence:** Not tested.
- **Root cause: NOT ESTABLISHED**.
- **Impact/severity:** High if unsupported figures are returned as factual.
- **Corrective action:** Add negative retrieval tests for absent metrics, incomplete date ranges, and irrelevant documents.
- **Retest criteria:** Each case returns a clear insufficiency response and no fabricated numerical answer or source.

FA-08 — Unsupported or oversized upload

- **Scenario:** Upload an unsupported file type or a file exceeding the configured limit.
- **Expected:** The application rejects the file safely and gives a clear error without storing a partial or unsafe artifact.

- **Actual/evidence:** Not tested in this report.
- **Root cause: NOT ESTABLISHED**.
- **Impact/severity:** Medium to high depending on exposure and file-processing behavior.
- **Corrective action:** Test MIME/type and extension validation, size limits, partial-write cleanup, and boundary-size files.
- **Retest criteria:** Supported files within limits succeed; unsupported and oversized files fail with controlled responses and no unintended persistent artifacts.

6. Limitations and Risks

The following are based on prior project notes/code-review observations and should be reconfirmed against the current working tree:

1. **Client-level authorization:** Shared workspace login may not provide tenant-specific isolation if the application is exposed to multiple clients.

2. **Upload size enforcement:** A possible gap exists when request size is checked using `Content-Length` without an independent body limit.

3. **Session deployment model:** In-memory sessions are process-local and may behave inconsistently across restarts or multiple workers.

4. **Sign-out state consistency:** The frontend may visually sign out even when the server request fails.

5. **Evaluation evidence gap:** Synthetic CAGR, retrieval grounding, upload edge cases, cross-client isolation, and dataset cleanup have not been independently verified for this report.

6. **Deployment readiness:** Available evidence is insufficient to declare the project production-ready.

7. Recommendations and Next Steps

Priority	Recommendation	Severity	Effort
P0	Run the full existing test suite locally and preserve actual output	High	Low

P0	Execute client-isolation tests with two synthetic clients and verify server-side authorization	High	Medium
P0	Verify upload size limits for absent/misleading `Content-Length`; enforce a hard body limit if needed	High	Medium
P1	Add deterministic CAGR and missing-source-data tests against synthetic fixtures	High	Low–Medium
P1	Verify sign-out failure handling and session expiry/restart behavior	Medium	Medium
P1	Test malformed requests and unsupported/oversized uploads	Medium–High	Medium
P1	Verify dataset loader repeatability and cleanup safeguards in an isolated disposable environment	Medium	Medium

P2	Document setup, test commands, synthetic-data preparation, expected API errors, and known limitations	Medium	Low
P2	Repeat `git diff --check`, review `git status`, and ensure no secrets or generated runtime state are included	Medium	Low

8. Conclusion

Project Atlas has a documented modular architecture combining a FastAPI backend, structured database access, retrieval functionality, and a multi-component analysis workflow. Prior project notes record a successful backend test run of 28 tests, but that result has not been independently rerun as part of this report.

This report does **not** claim that all required scenarios passed. Direct runtime validation of authentication, client isolation, source grounding, synthetic CAGR, uploads, frontend error handling, and dataset cleanup remains blocked or unrun. Architecture-level concerns are recorded as risks or suspected issues rather than confirmed runtime defects.

Readiness assessment: Suitable for continued local development and a carefully scoped demo after the current test suite is rerun. Production or multi-tenant readiness is not established. Complete the P0/P1 validation items, update this report with actual outputs and final counts, and review remaining failures before recruiter submission.

Appendix A — Local verification checklist

From `C:\\Users\\hp\\Desktop\\project atlas`, run the repository's documented commands in the existing virtual environment. A typical starting point is:

```powershell
.\\.venv\\Scripts\\python.exe -m pytest backend/tests -q
git diff --check
git status --short
```

Then run only safe evaluation scripts identified during code inspection. Do not run destructive cleanup against the working database. Update the results table, summary counts, and analysis only from observed evidence.